

DEFINITION BREAKER

A puzzle-adventure where the rules are physical objects

Observe. Rewire. Preview. Verify. Commit.

GUARD

when is change valid?

RULE OBJECT

LINEAGE

remember exactly what changed

Concept profile	DB-0.1
Document type	Game concept + technical design blueprint
Prepared for	Tom Klootwijk
Date	28 August 2026

Evidence boundary. This document is an original application design inferred from the supplied UGTS reports. The reports do not claim that *Definition Breaker* is already implemented.

HIGH-LEVEL PRODUCT DECISION

Executive Decision

Make the substrate's most unusual capability the player's main verb:

Definitions, operation order, compatibility, guards and transitions become carryable, connectable world objects. The player can propose radical changes, but the world changes only after a deterministic verification and commit.

DECISION**GO for a 2D systemic puzzle-adventure vertical slice.**

The concept is distinctive, technically grounded, and small enough to prove through the existing vector-game runtime before pursuing native 3D, VR or multiplayer.

GENRE

Systemic puzzle-adventure with light real-time hazards

PRIMARY FORMAT

Single-player, offline HTML5; desktop first

SIGNATURE MECHANIC

Physically rewire typed rule objects and commit verified world patches

CORE EMOTIONAL ARC

Confusion -> understanding -> controlled transgression -> mastery

MVP PROMISE

A 20-30 minute, 7-room vertical slice with deterministic replay

Why it is cool

Instead of finding a key, the player can change what "key," "door," "open," or "before" means - while the game visibly proves whether the edit is legal.

Why it is playable

The player manipulates a bounded subgraph with typed sockets, budgets, preview and readable rejection reasons. It is tactile puzzle solving, not arbitrary coding.

Why UGTS fits

The source line already separates definitions, instances, support, compatibility, guards, commits, patches and lineage, and 3.9 supplies the practical game shell.

Source-supported foundation. Content-addressed definitions and explicit operation order come from 3.6; event-authoritative proposal/commit and replay from Two Hands 3.0; vector gameplay, input, collision, audio and offline export from 3.9.

[S3 pp. 5-13](#)[S4 pp. 3-13](#)[S6 pp. 3-15](#)

SOURCE SCREEN AND EVIDENCE DISCIPLINE

Suitable Substrate Versions

The game uses the versions below according to their actual documented strengths. “Core” means required for the first playable; “Optional” means a later editor, metagame or rendering extension; “Excluded” means the source is domain-specific and would add no useful gameplay authority.

Source	Status	Contribution to Definition Breaker	Design boundary
UGTS-GN 1.1	CORE	Canonical support -> compatibility -> guard -> event -> transition -> lineage order; narrow one-bit semantics; precision and memory rules.	Packed flags never become complete state; performance requires equal-error measurement.
UGTS-KC 2.0	CORE	Pattern/field rules, route identity, winding, hybrid automata, event degeneracy and equal-time conflict concepts.	Topology is explicit routing, not physical magic; numerical status stays visible.
KC Two Hands 3.0	CORE	Stable nodes/assets, physical manipulation model, proposal verification, deterministic commit, checkpoints, replay and reason codes.	VR, general physics and live networking remain later work.
UGTS-KC 3.6	CORE	Definitions as first-class data, explicit dependencies, SHA-256 content addresses, operation order, definition queries and typed hinges.	References must resolve; cycles and hidden state are rejected unless a puzzle explicitly models them.
UGTS-KC 3.6.2 SCLP	CORE+	Winding, reflective wrapping, interval guards, bounded grammar, finite seed/index codecs and constraint-release puzzles.	A 64-bit key is an index, not a whole world or infinite detail.
UGTS-KC 3.9	CORE	Vector assets, multi-device input, fixed-step game world, collision, animation, tilemaps, procedural audio, saves and self-contained HTML5.	Native 3D, skeletal animation, multiplayer and general rigid-body physics are not claimed.
3.9.3 IQ Field	OPTIONAL	Rich 2D/3D field primitives, CSG, domain repetition and bounded procedural signals for later rule volumes and boss arenas.	Iterative queries require explicit budgets and status; presentation is never authority.
KC Atlas / 4.0 ledger	OPTIONAL	Evidence-bearing rule revisions, unknown states, proof trails and community-puzzle audit bundles.	Useful for workshop publishing, not required for moment-to-moment play.
3.6.3 SARA	EXCLUDED	Wallet-standard and authorization profile; no unique game mechanic needed by this concept.	Do not import cryptographic/wallet scope into the game merely because it is referential.



DEFINITION BREAKER

WORLD, ROLE AND PROMISE

High Concept and Player Fantasy

The premise

The player is a **Breaker**, a maintenance operative inside the Definition Stack: a city whose architecture is compiled from visible rules. After an event called **The Unresolved Cycle**, definitions detach from their instances. Doors inherit gravity, corridors forget which sheet they occupy, alarms and bridges commit contradictory patches, and some rooms depend on themselves.

The Breaker carries a Rule Lens and a portable Commit Bench. They can pull authorized rule objects from sockets, route typed cables, reorder phases, insert hinges, change topology ports and remove constraints. But raw manipulation is only a proposal. The world accepts a change after the substrate proves the edit is well-typed, supported, compatible, numerically safe, invariant-preserving and conflict-resolved.

“You are allowed to break the rule only after you understand exactly what the rule is.”

Player fantasy

- See the hidden logic that makes a world behave.
- Touch and rearrange that logic as tangible machinery.
- Use rejection, replay and lineage as investigative tools.
- Turn an impossible room into a valid new world - without cheating the authority model.

UNIQUE SELLING PROPOSITION

The laws of the level are inventory items.

Definitions are not abstract menu entries. They have mass, sockets, phase labels, dependencies, seals and history. Carrying a rule from one machine to another can be as meaningful as moving a crate.

Not a coding game

Players do not write arbitrary text. They manipulate a constrained, visual and tactile rule graph with authored choices, typed connectors, budgets, previews and undo.

Not pure puzzle-menu play

The changed definitions immediately alter traversal, hazards, NPC behavior, topology, timing and object ownership in the physical room.

Not consequence-free reality editing

Every accepted change leaves a lineage record. Later rooms may inspect what the player changed, in which order, and under which policy.

Game-design inference. The narrative, Breaker role and city are original design choices. Their mechanical credibility comes from the substrate's separation of definitions, instances, verified events, patches and lineage. [S4 pp. 3-6](#) [S3 pp. 6, 11-13](#)

THE GAME IN ONE CYCLE

Player Verbs and Core Loop



Inspect

Reveal stable IDs, type signatures, dependencies, operation phase, compatibility tags, guards, invariants and lineage.

Manipulate

Carry, socket, cable, rotate, reorder, branch, gate, hinge, wrap, lock, release or quarantine a bounded rule object.

Reason

Scrub a deterministic preview. Compare current and proposed state. Read rejection reasons as clues.

Exploit

Commit the valid patch, traverse the altered room, evade hazards, recover definitions and preserve the needed lineage.

Moment-to-moment rhythm

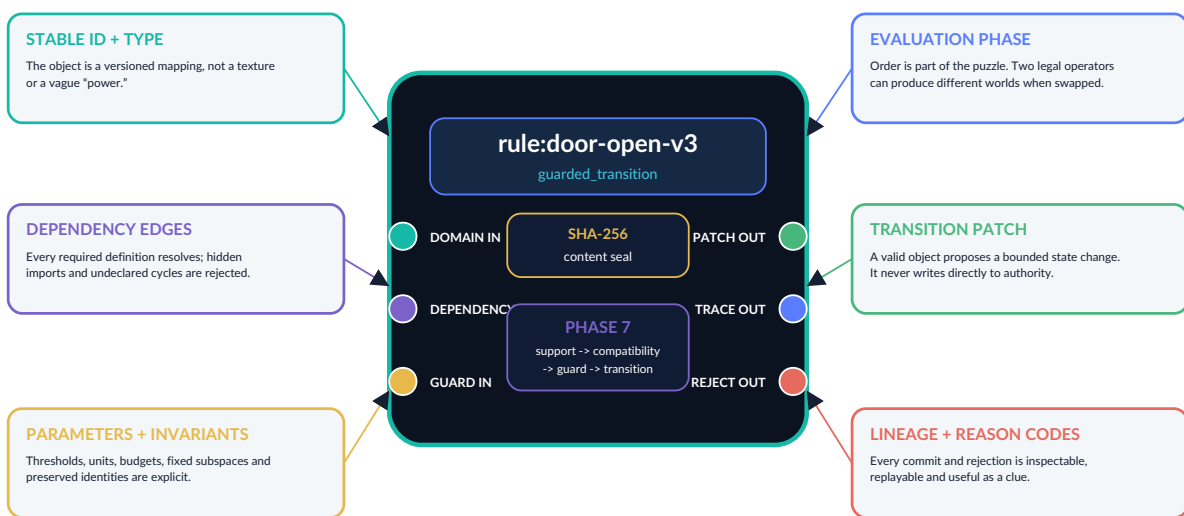
1. Enter a room and discover the physical symptom.
2. Use the Rule Lens to expose the relevant subgraph.
3. Find or earn access to a small set of rule objects.
4. Build a candidate edit on the Commit Bench.
5. Preview consequences and inspect trace/rejection.
6. Commit, then execute the traversal or timing challenge.

What produces mastery

- Recognizing type and dependency patterns.
- Predicting non-commuting operation order.
- Distinguishing coordinates from identity.
- Understanding crossing, touch, tangency and persistent constraints.
- Planning around deterministic conflict resolution and replay.

DATA MODEL TRANSLATED INTO GAME LANGUAGE

A Rule Object Is a Physical Definition



Non-negotiable design rule. The physical prop is a presentation of a canonical definition record. Picking it up does not change its identity; editing its parameters creates a versioned definition or a patch. A visual appearance, cable or icon never silently becomes the authoritative semantics. [S4 pp. 3-7](#) [S3 pp. 5-8](#)

WHAT THE PLAYER CAN CARRY

Rule Object Taxonomy

Literal Tile

Number, vector, tag, threshold, duration, phase, sheet or policy value. Small, stackable and easy to duplicate when allowed.

Typical puzzle: choose the right threshold without shrinking the event margin below safety.

Operator Core

A typed mapping with domain, codomain, parameters and capabilities: transform, compare, combine, project, integrate, route.

Typical puzzle: substitute an operator whose output actually matches the next socket.

Hinge

Continuous pivot, discrete parity selector or zero-magnitude connector with nonzero structural role.

Typical puzzle: insert a connector that changes graph order without changing numeric value.

Support Volume

Local admission region such as sphere, cone, shell, tile region or finite field support.

Typical puzzle: prevent a global rule from affecting unrelated objects.

Compatibility Seal

Sheet, phase, mode, orientation, ownership and application tags that determine whether supported states may couple.

Typical puzzle: make two co-located systems genuinely compatible.

Guard Blade

Boundary, threshold or relation with accepted statuses, confidence floor, error interval and hysteresis.

Typical puzzle: distinguish a true crossing from a tangency or chatter.

Transition Key

An atomic state, route, topology or dynamic reset applied only after verification.

Typical puzzle: change the requested patch without breaking stable node identity.

Topology Port

Explicit gluing, wrap, orientation transport, sheet update and winding lineage.

Typical puzzle: select a half-turn bundle or a genuinely reflective quotient.

Schedule Rail

Stores the actual evaluation/composition order. Blocks on the rail do not commute unless their definitions say so.

Typical puzzle: reorder translate, rotate, gate and commit.

Constraint Shackle

A maintained relation over an interval; removing one declared row may add a legal degree of freedom.

Typical puzzle: release exactly one motion without turning contact into infinite events.

Lineage Seal

Pre/post state hashes, committed event, source, branch and history reference.

Typical puzzle: preserve a required route history while changing position.

Quarantine Cage

Contains unresolved references, invalid hashes, forbidden cycles, overflowed grammar or incompatible definitions.

Typical puzzle: repair the graph rather than bypass validation.

Presentation inference. Names such as “Guard Blade” and “Lineage Seal” are game-language skins over source-supported record classes; the underlying semantics stay typed and inspectable.

MAKE FORMAL STATE READABLE AND TACTILE

The Commit Bench and Rule Lens

Rule Lens: understand the current world

RULE LENS / TARGET: bridge_17	
Instance	node:bridge_17
Definition	geom:hinged_bridge-v2
Phase	4 transform
Support	room_03 only
Conflict	translate precedes rotate
parse -> resolve -> construct -> transform -> support -> compatibility -> guard -> transition -> lineage	

The lens has three layers: **simple** (icons and plain language), **trace** (ordered operations and reasons) and **definition** (full typed record for expert players and workshop authors).

Physical input

- **Mouse/touch:** drag blocks, cable sockets, scrub preview, long-press to inspect.
- **Gamepad:** radial selection, snap-to-socket, shoulder-button phase movement.
- **Two-hand future:** stretch the bench, align ports and twist a rule cluster using the 3.0 bimanual transform.

Commit Bench: edit a candidate world

CANDIDATE PATCH / NOT AUTHORITATIVE	
3 CONSTRUCT	bridge
4 TRANSFORM	rotate 90 -> translate +2
5 SUPPORT	room_03
6 COMPAT	player + bridge
7 GUARD	angle >= 90
PREVIEW HASH: 3bd1...	VERIFY COMMIT

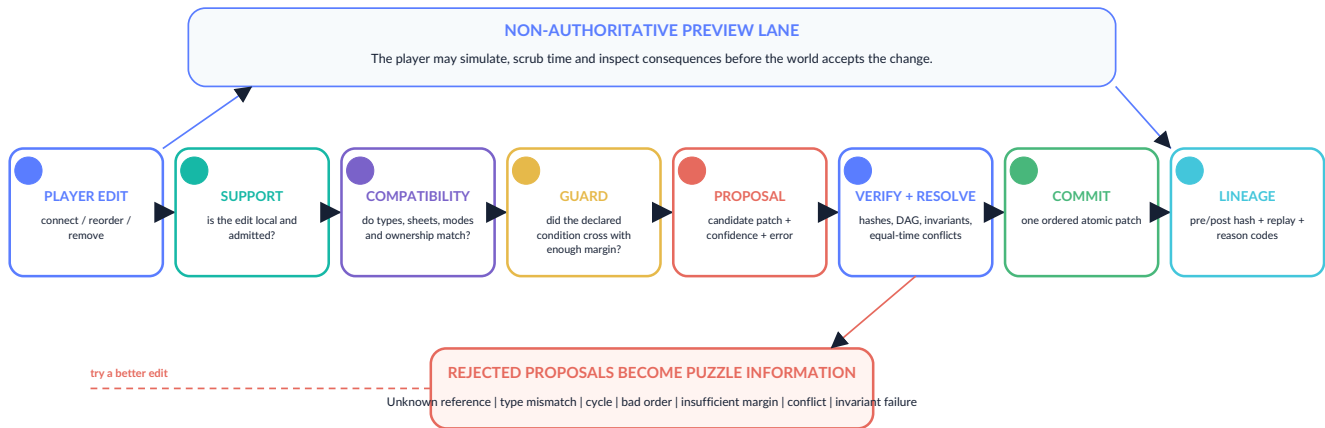
Usability protections

- Only sockets relevant to the current puzzle are exposed by default.
- Every block has shape, icon and text coding; color is never the only cue.
- Invalid cables refuse to snap and explain the domain/codomain mismatch.
- Preview is visually ghosted; the committed world is never ambiguous.
- Undo is checkpoint/replay, not a fragile reverse mutation.
- A hint can reveal one missing gate or one conflicting dependency without solving the entire graph.

Interaction boundary. The 3.9 action map supports keyboard, gamepad, pointer and touch. The richer two-hand manipulation is a later adapter target; the first release should not depend on native OpenXR. [S6 p. 6](#) [S3 pp. 11-12, 17](#)

THE GAME'S GOVERNING LAW

Every Edit Is a Proposal, Not a Cheat



Verification checklist

1. Definition IDs resolve and content hashes verify.
2. Dependency graph is acyclic under the selected cycle policy.
3. Socket domains/codomains and units match.
4. Edit lies inside an admitted support zone.
5. Sheet, phase, mode, orientation and ownership are compatible.
6. Guard status is accepted; numerical error is within margin.
7. Declared invariants and stable identities survive.
8. Same-time patch conflicts are ordered, merged or rejected.

Why this is fun rather than bureaucracy

The verifier is the puzzle's "physics." A rejection is concrete information: the player learns that a route is on the wrong sheet, a dependency is cyclic, a crossing is only a touch, or two valid edits conflict at the same tick.

Later levels deliberately require using the verifier as an instrument: construct a failed proposal to reveal a hidden phase, then repair it.

The pipeline preserves the recurring UGTS authority chain and the 3.0 proposal/commit split.

S1 pp. 2-4, 28

S2 pp. 11-12, 15

S3 pp. 5, 13

FAILURE BECOMES LEGIBLE

Preview, Rejection and Deterministic Explanation

Current world

The last committed state. Solid colors, collision and gameplay authority are active. Its canonical hash anchors saves, replay and puzzle goals.

CURRENT / 8ef1...

Candidate preview

A sandboxed simulation of the proposed graph. Ghosted geometry, predicted routes, contact traces and potential conflicts are visible, but nothing mutates authority.

PREVIEW / 3bd1...

Committed world

The verified patch is applied atomically; lineage and a replay record are appended. The new world receives a fresh state hash.

COMMITTED / a24c...

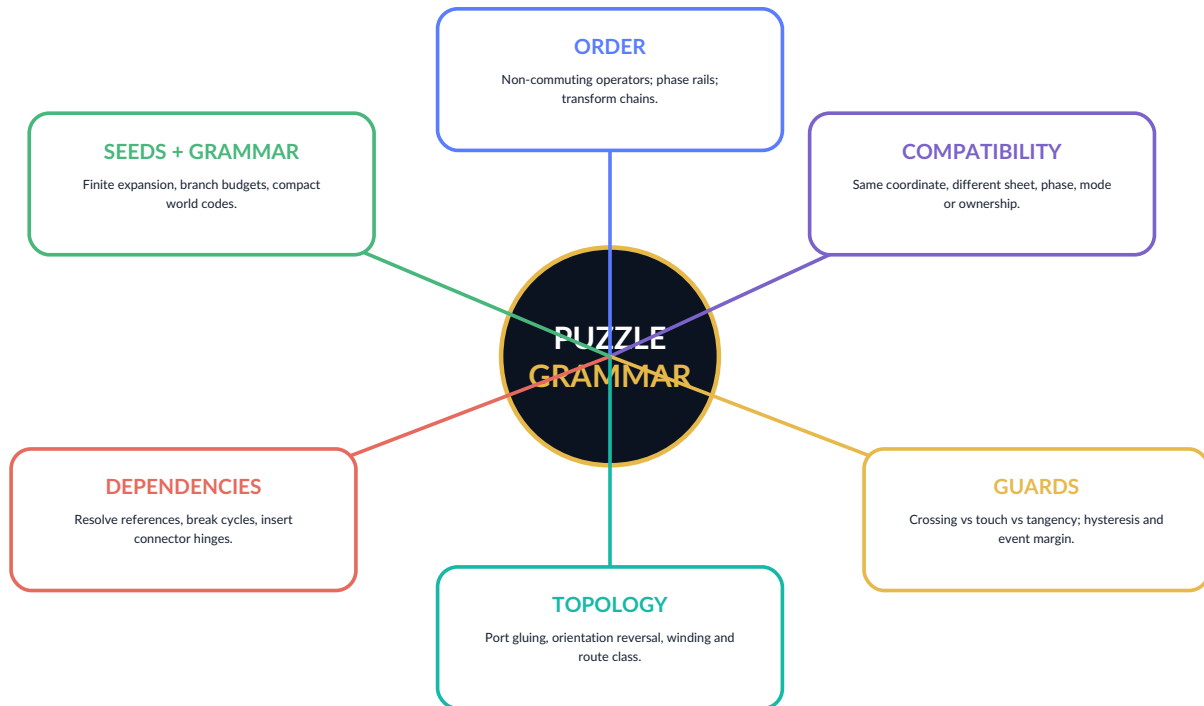
Rejection is a designed feedback channel

Reason code	Player-facing message	Puzzle meaning
unknown_reference	"This rule points to something that does not exist here."	Find the missing definition or replace the dependency.
type_mismatch	"A route is not a velocity."	Insert a projector/adaptor or use the correct socket.
dependency_cycle	"The door waits for the power that waits for the door."	Break the cycle with a latch, initial literal or connector hinge.
incompatible_state	"These objects overlap, but they do not occupy the same state sector."	Resolve sheet, phase, mode, orientation or ownership.
guard_ambiguous	"The relation interval touches both sides of the boundary."	Increase margin, improve the measurement or choose a safer guard.
patch_conflict	"Two valid rules request different values at the same event time."	Set priority, prove commutativity or prevent one proposal.
invariant_failure	"The edit would erase a stable identity or break topology."	Change the patch, not the identity; preserve required incidence/gluing.
budget_overflow	"This grammar would grow beyond the room's declared limit."	Reduce depth/branching or reuse a shared definition.

Determinism boundary. The design requires stable ordering and hash-checked replay in the selected runtime profile. It does not assume universal bit-identical floating-point behavior across every future CPU/GPU. [S3 p. 13](#)

REUSABLE FAMILIES FOR A FULL CAMPAIGN

The Six-Family Puzzle Grammar



Authored freedom, not arbitrary freedom

Each room exposes a deliberately small editable subgraph: usually 3-8 movable rule objects, 2-5 fixed definitions and one explicit goal. The space is rich enough for multiple valid solutions but bounded enough to read and test.

Solutions are proofs

A solution is not merely reaching the exit. The final state must satisfy the room goal, preserve required invariants and reproduce under deterministic replay from the room checkpoint.

THE FIRST HALF OF THE GAME'S LANGUAGE

Puzzle Families I-II: Order and Compatibility

I. ORDER / NON-COMMUTATIVITY

Two individually valid operators can produce different geometry or state when composed in a different order. The player physically moves blocks along a phase rail or rotates a transform chain.

Core patterns

- Rotate then translate versus translate then rotate.
- Apply a field morph before or after support clipping.
- Evaluate a hinge before a topology wrap so chirality is transported correctly.
- Resolve a guard before a dynamic reset rather than after it.
- Place a connector hinge that changes graph order while carrying zero numeric magnitude.

Readable cues

Ghost paths, phase-colored trails, before/after pivots, and a “first divergence” marker show where two traces become different.

Mastery challenge: construct two different valid orders that reach the same room goal but leave different lineage, then choose the one required by a later audit door.

II. SUPPORT / COMPATIBILITY

Spatial overlap is only the first question. Objects may still be separated by sheet, phase, route class, orientation, mode, policy or ownership.

Core patterns

- Two corridors cross in projection but occupy different sheets.
- A sensor sees the crate, but its support cone excludes the correct face.
- A hazard and player share a tile but are incompatible during a phase window.
- A rule is valid only inside a declared room support, preventing global side effects.
- A two-hand object remains owned after handover even though its position is unchanged.

Readable cues

The Rule Lens separates state sectors into layers, shows support volumes as translucent geometry, and marks incompatible overlaps with a dashed “no coupling” seam.

Mastery challenge: preserve two distinct states at one coordinate until a short compatibility window allows a controlled exchange.

Order as literal data and non-commuting hinges: [S4 pp. 5-6, 9, 13](#). Coordinate equality versus identity/compatibility: [S1 pp. 4-5, 18-19](#) [S2 pp. 4-5, 8](#).

TIME AND ROUTE BECOME MANIPULABLE

Puzzle Families III-IV: Guards and Topology

III. GUARD / EVENT TIMING

A value near zero is not automatically a crossing. The player must reason about direction, derivative, interval, hysteresis, priority and whether a relation should become a maintained constraint.

Core patterns

- Crossing, touch, tangency and coincidence produce different outcomes.
- A fast object needs a swept guard rather than a sampled overlap.
- Enter and exit thresholds prevent a switch from chattering.
- A persistent grab/contact must become a constraint mode, not infinite events.
- Two event-time intervals overlap and require priority, merge or conflict.

Signature room: "The Door That Opens Too Late." Move the predictive guard ahead of collision or add a certified swept relation so the bridge commits before the runner arrives.

IV. TOPOLOGY / ROUTE IDENTITY

Portals and wraps are explicit coordinate, orientation, sheet and lineage maps. The player manipulates route classes rather than relying on impossible visual tricks.

Core patterns

- Möbius-style wrap flips orientation after one traversal.
- Reflective Klein gluing is distinct from a simple half-turn bundle.
- Winding count distinguishes repeated phase coordinates.
- A self-crossing path may represent separate branches or route classes.
- Cell-complex edits must preserve incidence invariants.

Signature room: "Klein Freight." Route a marked crate through a reflective wrap, compensate its chirality, and deliver the correct orientation without merging two incompatible sheets.

Event degeneracy and persistent constraints: [S2 pp. 11-12](#) [S3 p. 11](#) . Explicit wrapping, winding and chirality: [S5 pp. 5-10](#) .

THE ADVANCED PUZZLE LANGUAGE

Puzzle Families V-VI: Dependencies and Seeded Grammar

V. DEFINITION GRAPH / REFERENTIAL CLOSURE

Definitions may depend on other definitions, but every reference must resolve, hashes must verify and the graph must obey its cycle policy. The player repairs the actual logic graph behind broken machines.

Core patterns

- Missing references create orphaned machinery.
- Duplicate IDs produce ambiguous sockets.
- A definition hash fails after an illicit parameter edit.
- A door-power-door dependency creates an unresolved cycle.
- A connector or latch breaks the cycle while preserving value and order.
- Replacing a definition changes every instance that references it; replacing one instance does not.

Signature room: "Gravity Is a Dependency." Detach a local room's gravity definition without changing the global definition used by the rest of the station.

VI. SEED / BOUNDED GRAMMAR / COMPRESSION

Rooms and challenge variants can be reconstructed from a seed, finite grammar and compact index, while external novelty and committed edits remain logged.

Core patterns

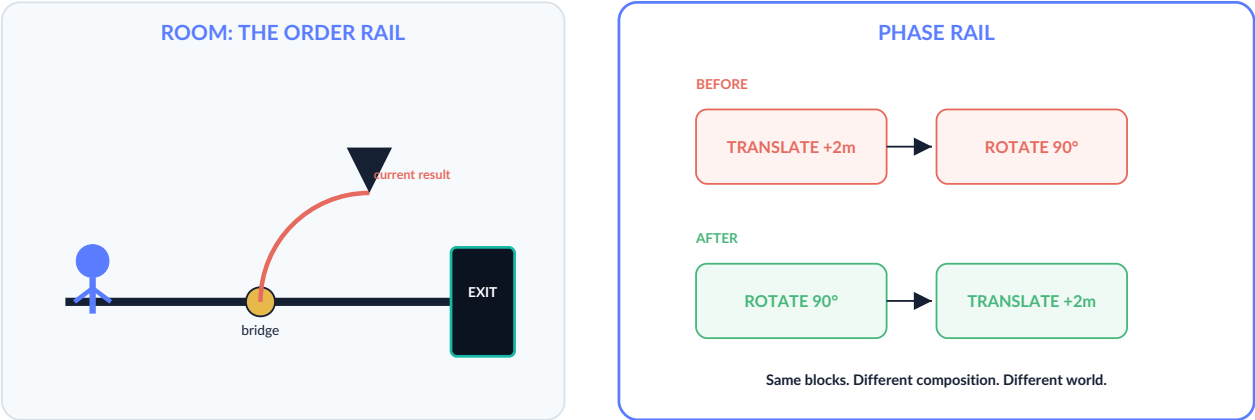
- Branch bits select one of two declared productions.
- Grammar depth, symbols and stack are budgeted.
- A 64-bit address chooses a quantized local cell/phase, not full state.
- Prefix refinement narrows a search region but does not directly halve physical coordinates.
- Seed + definitions reconstruct the baseline; player edits remain event history.

Signature room: "Worldseed 64." A map key locates the missing rule cache, but the player must recover the separate definition and lineage records to make the room whole.

Compression boundary. The game may exploit shared definitions, vector assets, bounded procedural generation and event-log reconstruction. It must never present nominal bit-width ratios as equal-information compression without a reconstruction and error contract. [S5 pp. 7-12](#) [S1 pp. 9-13](#)

DETAILED PLAYABLE SCENARIO 1

Example Room: The Order Rail



Room symptom

A hinged bridge rotates around the wrong point and misses the exit ledge. The same translation and rotation blocks are already present; the machine is “valid” but ordered incorrectly.

Intended discovery

Composition is non-commutative. The Rule Lens marks the first divergence between the current and candidate transform traces.

Solution

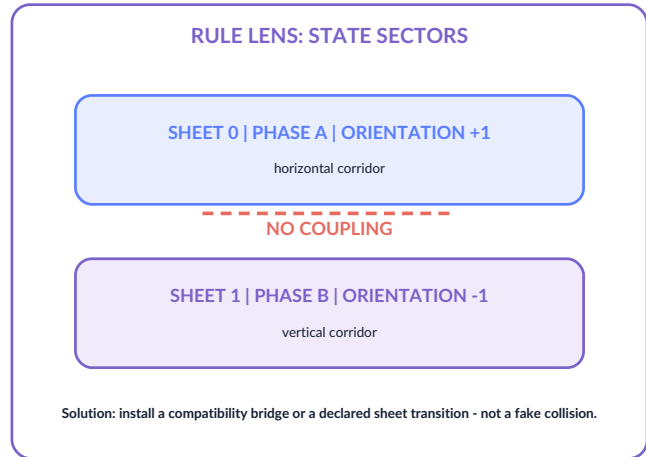
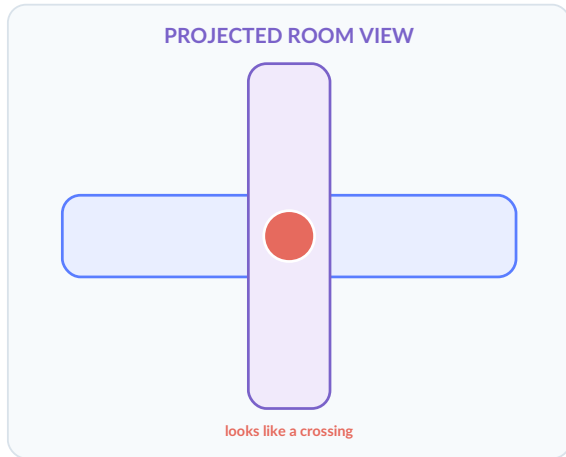
Move **ROTATE 90** before **TRANSLATE +2m** on the phase rail. Preview the pivot path, verify, then commit the transform-order patch.

Available objects	Hidden constraint	Failure feedback	Optional expert solution
Rotate, translate, pivot literal, room support, angle guard.	Bridge node identity and hinge pivot must remain stable.	“Invariant failure: pivot moved with the asset.”	Keep the original order but insert a compensating frame transform; produces a different lineage accepted by the goal.

Teaching purpose. This room turns 3.6's explicit non-commuting composition and 3.0's stable node identity into a readable first-act puzzle. [S4 p. 6](#) [S3 p. 6](#)

DETAILED PLAYABLE SCENARIO 2

Example Room: Same Place, Different Sheet



Room symptom

The player sees a corridor cross and expects to turn upward, but the avatar passes underneath. A power pulse appears to touch a receiver yet never couples.

Available edits

- Compatibility bridge: temporarily match sheet and phase.
- Declared sheet transition: move one route to the other sheet.
- Projection collision: tempting but invalid; it treats appearance as authority.

Intended solution

Install a compatibility window gated by the pulse phase, allowing energy to transfer for exactly one tick while the player routes through a separate portal. The world remains layered; the solution does not collapse two identities into one.

Failure feedback

incompatible_state: "Supported overlap found.
Coupling rejected: sheet 0 / phase A versus sheet 1 / phase B."

Conceptual boundary. The game may visualize sheets as layered corridors, but it should not imply that topology creates physical energy or that projected self-intersection is automatically contact. [S1 pp. 4, 18-19](#) [S2 p. 8](#)

DETAILED PLAYABLE SCENARIOS 3-4

Example Rooms: Missing Shackle and Klein Freight

THE MISSING SHACKLE

A suspended counterweight is constrained in two velocity directions. The exit mechanism needs one new legal degree of freedom, but simply deleting all constraints drops the mass and destroys the timing sequence.

Available objects

- Two constraint rows, one damped hinge, tangent projector, safety guard and reset.
- A “release one row” tool that exposes the nullity change.

Solution structure

1. Remove exactly the lateral constraint row.
2. Keep the radial row and tangent projector.
3. Attach the damped hinge so the arm swings rather than accelerates without bound.
4. Commit only when the safety guard certifies the new path.

Core lesson: constraint release increases legal motion; it does not automatically supply force, chaos or a trajectory.

KLEIN FREIGHT

A cargo crate must traverse a radial wrap. The displayed path looks like a half-turn, but the destination scanner requires a genuinely orientation-reversed state and a preserved winding label.

Available objects

- Source half-turn bundle, reflective Klein wrap, chirality automorphism, orientation seal and winding counter.

Solution structure

1. Select the reflective wrap rather than the orientation-preserving base rotation.
2. Let the wrap flip local orientation and hoop state.
3. Apply the chirality map to the crate's control convention.
4. Preserve the winding in lineage so the scanner distinguishes first and third arrival.

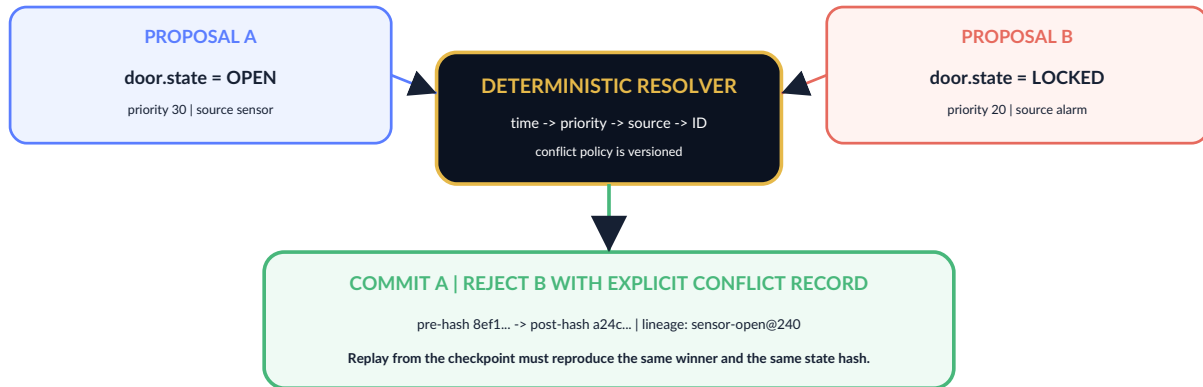
Core lesson: visually similar wraps can have different derivative/orientation maps and therefore different gameplay semantics.

Constraint row release and tangent projection: [S5 pp. 6-7](#). Half-turn versus reflective Klein profile and chirality: [S5 pp. 5-10](#).

DETAILED PLAYABLE SCENARIO 5

Example Room: Conflict at Tick 240

TICK 240: TWO VALID PROPOSALS, ONE CONFLICTING FIELD



Room setup

A sensor proposes **OPEN** while an alarm proposes **LOCKED** at the same fixed tick. Both proposals independently satisfy support, compatibility and guard checks. The door cannot accept both values.

Player task

Trace the previous loop, discover why the alarm fires, then either prevent one proposal, change the declared priority, or replace the patches with a commutative state representation such as an explicit *open-but-alarmed* mode.

Replay reveal

The game rewinds to the checkpoint and replays input and events. A ghost trace shows the same conflict and the same winner. The player can scrub directly to the first hash divergence after an edit.

Expert variant

Two players or two echoes each control one source. They must coordinate timing so the proposals no longer overlap, avoiding a priority hack.

Design inference. Turning deterministic conflict resolution into a time-loop puzzle is an application concept. The underlying ordered proposal, rejection, pre/post hash and replay mechanisms are source-supported. [S2 pp. 11-12, 21](#) [S3 p. 13](#)

[S6 pp. 6, 9, 12](#)

TEACH ONE DEEP DISTINCTION AT A TIME

Campaign Structure and Progression



The campaign increases expressive power, but never removes typing, budgets, preview or deterministic commit.

Chapter	Rule vocabulary	Mechanical escalation	Boss / set-piece
I. The First Hash	Stable ID, literal, type, instance versus definition.	Match sockets; repair one missing reference; learn preview/commit.	The Orphan - a machine with no definition.
II. The Phase Rail	Operation order, hinges, transforms, invariants.	Reorder blocks; preserve pivots; exploit connector hinges.	The Commutator - changes order every loop.
III. False Twins	Support, compatibility, sheet, phase, ownership.	Manage co-location without false coupling; timed compatibility windows.	The False Twin - two identical bodies with different lineage.
IV. The Event Horizon	Guard margin, crossing, touch, tangency, hysteresis, constraint.	Predict events; avoid chatter; convert persistent contact to a maintained mode.	The Chatter Engine - floods the room with invalid zero events.
V. Winding City	Port gluing, orientation, sheet change, winding and route class.	Route cargo, player and signals through different quotient rules.	The One-Sided Station - orientation is the actual health bar.
VI. Replay Court	Proposals, priorities, conflict, checkpoints, hashes, lineage.	Use past loops as proof; construct reproducible multi-event solutions.	The Auditor - accepts only solutions whose replay and lineage match.
VII. Definition Breaker	Dependencies, bounded grammar, seed/index, full authority chain.	Repair the Unresolved Cycle and decide which definitions the city should keep.	The Optimizer - tries to collapse identity, uncertainty and history into one bit.

TURN FORMAL DISTINCTIONS INTO MEMORABLE DRAMA

Narrative, Characters and Boss Logic

The thematic conflict

Definition Breaker is about the difference between freedom and unverified power. The city failed because its administrator, **The Optimizer**, attempted to compress every distinction - identity, state, uncertainty, route history and meaning - into minimal flags. The resulting "efficient" world is unable to explain itself and cannot safely change.

The Breaker restores not a single correct order, but an inspectable order: typed definitions, explicit dependencies, bounded edits and preserved lineage. The final choice can offer multiple valid city constitutions, each replayable and auditable.

Key characters

- **Mira / The Instance:** a recurring NPC whose body and asset change, but whose stable node identity and lineage persist. Teaches identity separation.
- **Latch:** a small connector-hinge companion that has "zero magnitude, nonzero structural role." It becomes a literal puzzle tool and comic voice.
- **The Auditor:** antagonist-turned-ally who refuses undocumented miracles; teaches proof, replay and reason codes.
- **The Orphans:** machines and citizens whose definitions are missing. Restoring them raises ethical questions about version and provenance.

THE COMMUTATOR

Reorders transform and event blocks. Damage is dealt by pinning a deterministic phase order long enough to make its own attacks miss.

THE FALSE TWIN

Creates co-located duplicates on incompatible sheets. The player must target lineage and compatibility rather than appearance.

THE CHATTER ENGINE

Oscillates around a guard boundary. The solution is hysteresis, hold time or a constraint mode - not faster button pressing.

THE ONE-BIT KING

Claims one selector contains geometry, state and identity. The arena is won by splitting those roles into explicit objects.

THE OPTIMIZER

Deletes uncertainty and lineage to make every patch "fast." The final fight uses evidence, conflict traces and reconstruction to prove why a smaller record can be less truthful.

Narrative inference. The characters and themes are original. They dramatize explicit source boundaries: one-bit narrow roles, coordinates not identity, packed hashes not complete lineage, and compression not correctness. [S1 pp. 10, 13, 15](#) [S5 pp. 11-14](#)

MAKE LOGIC INSTANTLY READABLE

Art Direction, Animation and Procedural Audio

Visual language

support

compatibility

guard

commit

rejection

trace

- **World:** clean vector architecture, dark substrate voids, luminous path networks and readable silhouettes.
- **Rule objects:** semi-transparent blocks with physical sockets, phase bands, content seals and animated dependency cables.
- **Preview:** ghost geometry, dotted collision volumes and alternate route overlays.
- **Topology:** sheet separation shown through parallax/layer offset rather than impossible camera tricks alone.
- **Definitions:** resolution-independent SVG/Canvas paths; no texture atlas required for the first slice.
- **Patterns:** Gielis, Lissajous, rose, spiral and periodic-field contours give machines distinct identities without sampled art.

Animation

Rule blocks pulse by state: idle, candidate, verified, rejected and committed. Keyframe clips, easing, ping-pong clocks, crossfades and property-path application are enough for the first release.

Audio grammar

Event	Sound logic
Support admitted	Short low click; one oscillator voice.
Compatibility complete	Second interval joins the first; consonance indicates matched tags.
Guard approaching	Pitch rises with normalized residual, but never pretends to be probability.
Verified proposal	Three-note cadence; still unresolved until commit.
Commit	Full chord plus percussive transient; lineage motif appended to music.
Rejection	Dissonant interval shaped by reason family, followed by a readable text cue.
Replay divergence	Beat sequence splits at the first mismatched hash.

AUDIO TARGET

The player should hear the authority chain.

Procedural oscillator/noise cues and beat sequences keep the package compact and make each verification stage musically legible.

Screen output

The first game can use the 3.9 Canvas renderer, gradients, transforms, z-order, camera, grid, vignette, particles and HUD. More advanced field visualization from the IQ Field profile is optional.

Vector, animation and procedural-audio support: **S6 pp. 5-7, 10, 14** . Pattern families: **S2 pp. 5-7, 16+** .

DEPTH WITHOUT EXCLUSION

Accessibility, Difficulty and Hint Design

Perceptual access

- Color + icon + shape + text for every rule class.
- High-contrast mode and reduced glow.
- Adjustable trace thickness and animation speed.
- Screen-reader labels for definition inspector and reason codes.
- No solution depends only on tiny moving particles.

Motor access

- Snap-to-socket and one-button cable routing.
- Full pause during rule editing in standard mode.
- Hold/toggle options for Rule Lens.
- Remappable named actions across keyboard, gamepad and touch.
- Undo to checkpoint without timing precision.

Cognitive access

- Plain-language layer before formal type detail.
- One new distinction per early room.
- Trace highlights the first divergence, not every intermediate value.
- Reason codes grouped into reference, type, compatibility, guard, conflict and invariant families.
- Glossary terms unlock in context.

Three difficulty profiles

Profile	Editing time	Inspector	Failure assistance	Best for
Guided	World pauses fully.	Plain language, relevant sockets only.	Highlights the first failed gate and suggests one next action.	Story players and accessibility.
Standard	World pauses at benches; hazards continue in late chapters.	Plain + trace layers.	Full reason code and before/after comparison.	Intended first playthrough.
Expert / Audit	Optional live edits and tighter budgets.	Full record, hashes and dependency graph.	No suggested action; deterministic trace remains available.	Optimization, speedruns and workshop validation.

Hint ladder

1. Symptom

2. Relevant phase

3. First failed gate

4. Missing rule class

5. Candidate wiring

6. Full solution

Design goal. The formal substrate should increase explainability, not create jargon walls. The expert layer is optional; the physical world and plain-language rejection should carry the main experience.

ADDITIVE SCHEMA, INSPECTABLE RECORDS

Technical Game Data Model

Project-level additions

The first implementation can extend the 3.9 GameProject additively rather than inventing a separate engine:

```
{
  "schema": "ugts-kc-definition-breaker-0.1",
  "rule_library": [...content-addressed definitions...],
  "rule_instances": [...physical blocks and fixed
sockets...],
  "edit_zones": [...what the player may manipulate...],
  "commit_policies": [...accepted statuses, margins,
conflicts...],
  "puzzle_goals": [...state, route, lineage and replay
predicates...],
  "scenes": [...ordinary 3.9 entities, tilemaps, art and
audio...]
}
```

Canonical rule definition

```
{
  "id": "rule:door-open-on-power-v1",
  "kind": "guarded_transition",
  "domain": "door_state x power_state",
  "codomain": "door_state",
  "evaluation_phase": 7,
  "dependencies": ["guard:power-positive-v1"],
  "parameters": {"threshold": 1.0},
  "invariants": ["door_node_id_stable"],
  "provenance": {"class": "game-authored"},
  "content_hash": "sha256:..."
}
```

Runtime records

RuleInstance

id, definition_ref, transform, socket_state, ownership, edit_zone, lineage_ref

RuleSocket

domain, codomain, phase, multiplicity, allowed_kinds, lock_state

RuleEditProposal

id, tick, editor, target, operation, definition_ids, expected_pre_hash, confidence, numeric_error, margin, lineage_label

RulePatch

add/remove/replace/reorder/connect/disconnect/parameter/topology/constraint operation

PuzzleGoal

required state predicates, route class, preserved invariants, forbidden reason codes, replay hash policy

RuleTrace

ordered phase outputs, first divergence, gate results, conflict decisions, pre/post hashes

Identity rules

- Definition ID != physical rule-block instance ID.
- Scene-node ID != asset ID != current coordinate.
- Content hash verifies canonical semantics but does not replace lineage.
- Replacing a referenced definition may update many instances; moving one block does not create a new definition.
- Rendered icon and color are downstream views of the record.

This schema is an application-level proposal derived from the 3.6 definition record, 3.0 scene/event records and 3.9 project model.

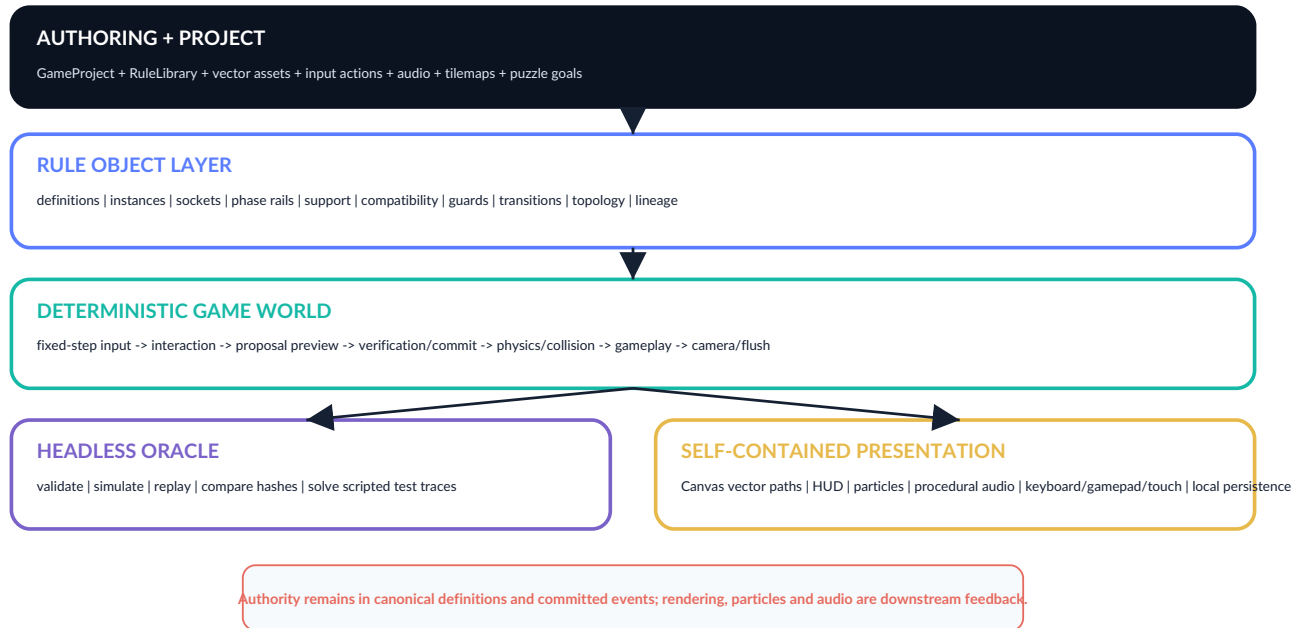
[S4 pp. 3-7, 11-12](#)

[S3 pp. 6-7, 19](#)

[S6 pp. 3-4](#)

ONE PROJECT, TWO EXECUTION TARGETS

Runtime Architecture and System Order



Suggested components

Transform2D, Body2D, Collider2D, RuleCarrier, DefinitionRef, RuleSocket, SupportGate, CompatibilityTags, GuardRelation, TransitionPatch, TopologyState, ConstraintMode, LineageRef, PuzzleGoal, InspectorHint.

Suggested fixed-step insertion

1. Resolve InputFrame.
2. Player controller and rule-object manipulation.
3. Assemble candidate graph and preview requests.
4. Verify/commit accepted rule proposals at a stable pre-physics phase.
5. Physics, bounds and collision.
6. Gameplay hazards, pickups and puzzle-goal checks.
7. Animation, lifetime and procedural feedback.
8. Camera, deferred destruction, snapshot/hash/save.

Implementation decision. Keep the rule system as custom, stably ordered systems inside the existing fixed-step world. Do not let the browser UI mutate entity dictionaries directly. All committed edits go through the same proposal and patch path used by headless tests.

SMALL, DETERMINISTIC AND HONEST

Persistence, Seeds, Compression and Performance

Save/reconstruction strategy

1. Store immutable/vector definitions once and reference them by stable ID/content hash.
2. Generate baseline room geometry from project seed and bounded grammar where reconstruction is deterministic.
3. Store current compact hot state: stable IDs, transforms, modes, phase/sheet/orientation, active constraints and lineage references.
4. Append authoritative player edits, topology changes, conflict decisions and external inputs as events.
5. Create checkpoints when replay cost or divergence risk exceeds the room budget.
6. Verify a loaded save by replaying from the nearest checkpoint and comparing the canonical state hash.

64-bit key use

Use a finite key as a local room/phase index or deterministic challenge code. It may accelerate lookup and prefix refinement, but must not stand in for stable identity, definitions, uncertainty, route state or full lineage.

Compactness advantages

- Vector paths and gradients replace texture atlases for the first release.
- Procedural oscillator/noise cues replace sampled effect files.
- Shared definition records prevent repeated rule payloads.
- Seeded room variation stores parameters and branch choices rather than materialized copies.
- Verified-event compaction avoids dense logging of unchanged candidates.
- Cold definitions/editor metadata stay outside the hot simulation record.

Targets to measure, not claims

Target	Initial design goal
Fixed-step rate	60 ticks/s target on ordinary desktop browsers; measure p50/p95/p99.
Room rule graph	Usually <= 64 active rule instances and <= 256 definition dependencies.
Grammar	Explicit depth/symbol/stack budgets; reject overflow before simulation.
Preview latency	Immediate for local edits; degrade by reducing horizon, not by hiding status.
Web package	Small enough for instant offline opening; exact byte target set after the first slice.

Precision rule. Packed/fixed-point or reduced records are accepted only when coordinate and guard error remain below the declared event margin and do not change ordering. Memory reduction is not correctness. [S1 pp. 9-15](#) [S5 pp. 8-14](#)

SHIP THE SIMPLEST DISTINCTIVE VERSION FIRST

Platform, Modes and Expansion Path

FIRST: OFFLINE HTML5

Desktop browser and static-file distribution using 3.9's Canvas/ Web Audio runtime.

- Keyboard, gamepad, pointer and touch.
- Headless Python validation and simulation.
- Single-file or split bundle.
- Local saves and best-solution records.

RECOMMENDED MVP

SECOND: NATIVE DESKTOP / ANDROID

Port the stable record and commit core only after the puzzle schema is proven.

- Native input and storage.
- Optional field/GPU presentation.
- Phone-scale touch Rule Bench.
- Reuse compact vector/seed assets.

AFTER VERTICAL SLICE

LATER: TWO-HAND / CO-OP

Use bimanual transforms and deterministic ownership for spatial rule manipulation.

- Local co-op “dual authority.”
- VR hand joints and handover.
- Server-authoritative workshop sessions.
- Replay and interest management.

NOT MVP

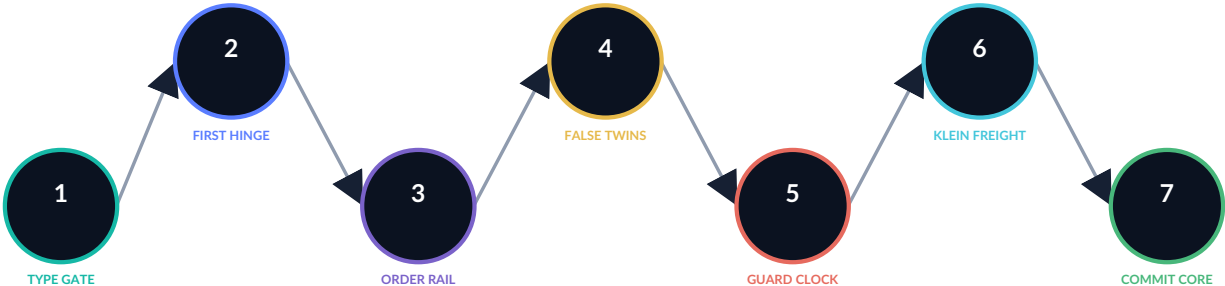
Game modes

Mode	Core promise	Substrate leverage
Campaign	Authored puzzle-adventure with narrative and bosses.	Full rule vocabulary, fixed-step runtime, checkpoints and progression.
Worldseed Rooms	Daily compact challenge code with verified solution replay.	Bounded grammar, 64-bit index, state hash and input/event recording.
Audit Trials	Optimize a room under rule-count, event-count or lineage constraints.	Reason codes, trace metrics, canonical hashes and deterministic conflicts.
Workshop	Author and share bounded puzzle graphs without arbitrary scripts.	Content-addressed definitions, schema validation, source/asset hashes; optional evidence ledger.
Dual Authority	Two players hold different rule classes and must coordinate commits.	Ownership, handover, proposals, same-time conflicts and replay.

Roadmap boundary. The supplied reports do not claim a finished multiplayer service, native OpenXR binding or general console/ mobile engine. Those remain explicit later engineering tasks. [S3 pp. 15-17](#) [S6 p. 15](#)

THE SMALLEST RELEASE THAT PROVES THE IDEA

Recommended Vertical Slice



VERTICAL SLICE TARGET

20-30 minutes | 7 rooms | 1 rule bench | 1 deterministic replay reveal | 1 topology boss | offline HTML5 build

Playable scope

- One compact facility wing, 7 rooms, 20-30 minutes.
- Player movement, dash, health/hazard fallback and camera from 3.9.
- Rule Lens with simple and trace layers.
- Commit Bench supporting connect, disconnect, reorder and replace.
- Rule objects: literal, operator, hinge, compatibility seal, guard and transition.
- One deterministic replay sequence and one same-tick conflict.
- One orientation-wrap boss using sheet/orientation state.
- Self-contained offline HTML5 build plus headless smoke trace.

Acceptance metrics

Gate	Pass condition
Clarity	New players solve the first three rooms without opening the formal record layer.
Authority	No UI or presentation path mutates canonical state outside verified commit.
Replay	Every accepted room solution reproduces its final state hash from checkpoint.
Reason codes	Every rejected edit reports one primary cause and exact target/reference path.
Boundedness	No room can create unbounded grammar, event chatter or dependency growth.
Delivery	Project validates, simulates headlessly and opens offline in a browser.
Fun	At least one room produces an "I changed the law" moment rather than a cable-matching chore.

VERTICAL-SLICE DECISION

Prove Definition -> Physical Object -> Preview -> Verified Commit -> Changed Traversal.

Do not begin with a general-purpose visual programming editor. Build seven authored rooms around a narrow rule taxonomy, then expand only after the loop is enjoyable.

BUILD, MEASURE, AND STOP WHEN THE PREMISE STOPS HELPING

Implementation Phases, Kill Criteria and Source Register

Implementation phases

0. Contract	Rule schema, canonicalization, definition hashes, dependency validation, proposal and patch records.
1. Tactile loop	Physical rule blocks, sockets, Rule Lens, preview state and verified commit in one test room.
2. Puzzle proof	Four rooms: type, order, compatibility and guard. Headless replay and reason-code tests.
3. Distinctive content	Topology room, conflict/replay room, boss, procedural audio and final art pass.
4. Productization	Campaign shell, accessibility, saves, challenge codes, build reports, manifests and offline packaging.

Kill or simplify when...

- Players must read raw JSON to understand normal rooms.
- The editable graph grows faster than support/compatibility can prune it.
- Invalid edits return generic failure rather than a useful reason.
- Visual preview cannot clearly distinguish candidate from committed state.
- Packed precision or cross-platform behavior changes event order.
- Topology is only cosmetic and never alters route, orientation or compatibility.
- Most content is cheaper and clearer as ordinary authored scripting.
- The four-room prototype feels like wiring homework instead of world transformation.

Source register

ID	Source and most-used pages
S1	UGTS-GN 1.1, pp. 2-5, 9-15, 18-19, 28: canonical authority, identity, packed records, precision, memory, one-bit boundaries.
S2	UGTS-KC 2.0, pp. 4-15, 21: pattern/field roles, route identity, hybrid events, degeneracy, memory and final definition.
S3	KC Two Hands 3.0, pp. 5-13, 15-19: scene identity, physical interaction, proposal/commit, replay, records and roadmap boundaries.
S4	UGTS-KC 3.6, pp. 1-14: content-addressed definitions, dependency graph, hinges, operation order, traces and kill criteria.
S5	UGTS-KC 3.6.2 SCLP, pp. 5-15: interval guards, winding, reflective wrapping, constraint release, bounded grammar and 64-bit layouts.
S6	UGTS-KC 3.9, pp. 3-15: vector art, input, animation, tilemaps, audio, fixed-step game world, persistence and HTML5 export.
S7	UGTS-KC 3.9.3 IQ Field, pp. 16, 23: optional richer field primitives and bounded procedural/query status.
S8	KC Atlas 3.9.3 / Spatial Evidence 4.0, pp. 4-7, 11, 17-18: optional evidence, revision, unknown-state and publication ledger for workshop content.

Final evidence boundary. This design preserves source terminology and documented limits, but all game loops, rooms, characters, bosses, UI, progression, targets and implementation sequencing are original application inferences. No novelty, patentability, commercial viability or completed-game claim is made by this PDF.

The strongest version of *Definition Breaker* is not a game where “anything is possible.” It is a game where every impossible-looking change becomes possible only after the player can explain it.